

OWASP
Sentinel:Context-
Aware Automated
Vulnerability
Scanning System



OWASP
SENTINEL
CYBERSECURITY & DEFENSE

Abstract

As the internet gets more complicated web application vulnerabilities are still a threat to global cybersecurity. Many small organizations, students and people who are new to the field struggle with the resource-heavy tools used to find these vulnerabilities.

This project is about creating a system called OWASP Sentinel.

It is designed to be lightweight, smart and aware of its surroundings.

It helps find flaws in web applications specifically those listed in the OWASP Top 10 for 2025.

OWASP Sentinel works differently from tools.

It doesn't just try everything to find a vulnerability.

Instead it looks at the web applications technology form structures and server behavior.

Then it uses this information to target its search for vulnerabilities.

The system is written in Python. Uses `asyncio` for fast processing.

Some key features of OWASP Sentinel include:

It can handle HTTP sessions well even if they are not responding away.

It has engines for detecting different types of vulnerabilities such as:

- + SQL Injection (SQLi)

- + Cross-Site Scripting (XSS)

- Server-Side Template Injection (SSTI)

Local File Inclusion (LFI)

It stores its findings in a way using SQLite.

The project used a mix of research methods.

It combined gathering requirements with testing on platforms like DVWA and OWASP Juice Shop.

The results show that OWASP Sentinels context-aware approach works well.

It reduces positives and improves confidence in its findings.

This makes it a good option for organizations, with resources to use for proactive security auditing.

Table of Contents

1.Introduction

1.1:Project Context

1.2:Problem Statement

1.3:Aim and Objectives

1.4:Rationale

1.5:Scope and Limitations

1.6:Thesis Structure

2:Background and Literature Review

2.1. Evolution of the Web Threat Landscape

2.2. The OWASP Top 10 Framework

2.3. Critical Analysis of Existing Scanners

2.4. Context-Aware vs. Signature-Based Paradigms

2.5. Scholarly Perspectives on Automated Diagnostics

3. Research Methodology

3.1. Mixed-Method Rationale

3.2. Requirements Elicitation Strategy

3.3. Secure Development Lifecycle (SDL)

3.4. Empirical Evaluation Framework

4: System Analysis and Design

4.1. Requirements Engineering

4.2. Functional and Non-Functional Requirements

4.3. High-Level Architectural Design

4.4. Component Interaction and Data Flow

4.5. Database and Persistence Schema

5. Implementation

5.1. Technical Stack and Library Rationale

5.2. Resilient Core Session Management

5.3. The Contextual Identifier & Reconnaissance Engine

5.4. Vulnerability Detection Module Logic

5.5. Reporting and Data Persistence

6. Testing and Evaluation

6.1. Verification and Validation Strategy

6.2. Benchmarking against Standard Vulnerable Targets

6.3. Accuracy and Resilience Performance Analysis

6.4. Comparative Evaluation with Industry Tools

7. Discussion and Critical Reflection

7.1. Analysis of Technical Outcomes

7.2. Critical Self-Appraisal of the Development Process

7.3. Ethical Considerations and Strategic Impact

8. Conclusion and Future Work

8.1. Summary of Achievements

8.2. Strategic Recommendations

8.3. Future Horizons

9. References

10. Appendices

Chapter 1: Introduction

1.1 Project Context

We live in an age of rapid digitization. New web applications are built every second to power mission critical systems, facilitate human connection, and conduct global business. With this growth of the internet comes a larger attack surface. Web security is now one of the biggest priorities across organizations of all sizes. In the past, discovering common vulnerabilities like those listed by OWASP has required niche security expertise as well as bulky and cost-prohibitive automated solutions. The cybersecurity industry has come a long way, but there has always been a gap between advanced scanners and the security professionals who could benefit from them the most: those in SMEs and universities.

Traditional vulnerability scanners use heavy, context-blind fuzzing. This approach works well for organizations with resources to burn, but will likely hit rate-limits or produce copious amounts of "noise" (false positives). Modern websites use complex JavaScript frameworks and server-side templating, there has never been a more critical need for "smarter" automation that can observe before it acts.

1.2 Problem Statement

Although scanners like Burp Suite and OWASP ZAP have great features and capabilities, they also come with obstacles to operation that prevent smaller organizations and inexperienced students from utilizing them. This can be attributed to a variety of factors.

- 1.They require steep learning curves to understand all of the features and how to configure them for your target.
- 2.Many enterprise grade scanners require large amounts of memory/cpu overhead.This makes it difficult to operate

them on home machines or certain cloud services.

3. Tools can be brittle and unable to handle web applications with certain defensive measures in place. This includes failing to retry when HTTP 503 errors are returned or being blocked by rate limiting (HTTP 429).

1.3 Aim and Objectives

Goal: Design, develop and experimentally validate OWASP Sentinel, a context aware automated vulnerability scanner with usability and depth for today's OWASP threats.

Objectives:

1. Review literature of past/current web vulnerabilities as well as current automated scanner technology.

2. Gather requirements from SecOps professionals to learn about their pain points with current solutions via semi-structured interviews.

3. Architect a scalable asynchronous python solution fit for low-latency operations on lightweight machines.

4. Develop contextual awareness to enumerate technology used on the server side then utilize that info to alter payloads.

1.4 Rationale

The reason we made OWASP Sentinel is because we want to make security testing available to everyone. We call this "Democratizing Security Diagnostics". When even small mistakes can cause problems like data breaches people should be able to test for problems early on without having to deal with complicated or expensive tools. OWASP Sentinel uses something called Intelligence. This means we do not just send a lot of tests to see what works. Instead we try to figure out which tests are most likely to succeed. This way we do not overload the network. Put too much strain on the servers. It also gives developers,

who may not be experts, in security results that're easier to understand and use.

1.5 Scope and Limitations

The OWASP Sentinel tool is made to find the critical vulnerabilities in the OWASP Top 10 list from 2023 and 2025. It focuses on three things: Injection, Broken Access Control and Security Misconfigurations. The OWASP Sentinel tool is a kind of scanner that does not hurt the data it is looking at.

The OWASP Sentinel tool has some limitations.

The OWASP Sentinel tool does not handle authentication. It can keep a user logged in for a time but it does not work with special authentication methods like multi-factor authentication or CAPTCHA.

The OWASP Sentinel tool has trouble with client-side applications. It is good at looking at HTML forms. It has problems with applications that use React or Vue to generate their pages.

The OWASP Sentinel tool is not good, at getting around strong firewalls. It tries to be sneaky. It does not have special techniques to evade advanced Web Application Firewalls.

1.7 Formal Research Questions

To help us really understand what is going on we came up with some questions to guide our study.

1 We want to know how much better a context-aware reconnaissance phase is at finding vulnerabilities compared to using non-contextual fuzzing.

2 We are looking into how we can use architectural patterns to reduce the amount of computing power needed for security diagnostics on devices that do not have a lot of resources.

3. We need to figure out what is stopping people who're new to the field, from using the same scanning tools that big companies use and how we can make a simpler version that they can use instead.

1.8 Strategic Alignment and Graduate Attributes

The people who made OWASP Sentinel made sure it fits with the UWS Graduate Attributes.

Universal: OWASP Sentinel is for a problem that affects the whole world. It helps people think about security first and it does not matter where they are.

Work-Ready: OWASP Sentinel uses things that people in the industry use like asyncio and requests. It also follows the OWASP Top 10 framework. This means that OWASP Sentinel is ready for people who do security work.

Successful: OWASP Sentinel was tested against things that are supposed to be weak like Juice Shop and DVWA. It did a job, which means the ideas, behind OWASP Sentinel are good and it can help people who do automated diagnostics.

1.9 Thesis Structure

This report is divided into eight parts. After this introduction the security situation and

ways we find problems now are looked at closely in Chapter 2. Then in Chapter 3 the way we did the research for the project is explained. The security system and how it is designed are talked about in Chapter 4 and how it was actually built is explained in Chapter 5. What happened when we tested it and how well it worked is summarized in Chapter 6. The security project and what it means are thought about carefully in Chapter 7 and in Chapter 8 the whole study is wrapped up and ideas, for future research are given.

1.10 Summary of Chapter

This first chapter gives us the information we need to understand the OWASP Sentinel project. It talks about the problems we have with making web security tests easy to use. The OWASP Sentinel project is trying to solve these problems. This chapter also tells us what the OWASP Sentinel project is trying to do and what it wants to find out. It explains why the OWASP

Sentinel project needs to be good at understanding the situation it's in and automating things. This chapter also gives us an idea of what the rest of the study will be about and gets us ready, for the part where we look at what other people have written about the OWASP Sentinel project.

Chapter 2: Literature Review

2.1 Evolution of the Web Threat Landscape

The web application security scene has changed a lot since the World Wide Web started. At first people focused on keeping the network safe with things like firewalls and systems that detect intruders. These systems looked for things in the way data was sent.. As web applications became more interactive and allowed people to do more things the security threats moved to the application itself.

Some researchers like Fonseca and his team found out that problems with the application, like SQL Injection and Cross-Site Scripting can get around the network security. This is because they use the way that data is sent over the internet. Over the years these types of attacks have become more complicated. At first SQL Injection attacks were pretty simple. Now they

can be really sneaky and even steal data without being detected. Also with the rise of frameworks that run on the client side there are new types of Cross-Site Scripting attacks that target the way the website is built rather, than just the database. This shows that attackers are now going after the underlying code and templates of the website, which's a bigger problem. Web application security is still a big issue and web applications are still being targeted by SQL Injection and Cross-Site Scripting attacks.

2.2 The OWASP Top 10 Framework

The Open Web Application Security Project or OWASP for short has a list called the Top 10 that helps people find and group the security risks for web applications. This list is not a collection of vulnerabilities it is a way to prioritize risks based on a lot of data from people who work in security as the Open Web Application Security Project noted in 2025.

If you look closely at the lists from 2021, 2023 and 2025 you can see that the Open Web

Application Security Project has changed how they group these risks. They used to focus on the symptoms. Now they look at the root causes. For example the category called Broken Access Control includes a lot of different vulnerabilities that used to be separate. This is because Broken Access Control is a big risk for web applications. The Open Web Application Security Project also has a category called Insecure Design, which means that security should be part of the development process from the start not something you add later. The OWASP Sentinel project is focusing on two areas: A03:2025 Injection and A05:2025 Security Misconfiguration. The Open Web Application Security Project chose these areas because they can be automated effectively and they pose a risk, to the entire system if something goes wrong.

2.3 Critical Analysis of Existing Scanners

Vulnerability scanning has become big business it is now worth billions of dollars.. The thing is, the companies that make these tools and the people who make free

tools are still very different, from each other. Vulnerability scanning is still split into two groups, the tools and the open-source tools and they do not seem to be working together. Vulnerability scanning is a thing and it is strange that the commercial and open-source tools are not more similar.

2.3.1 Enterprise Solutions (Burp Suite, OWASP ZAP)

People use tools like Burp Suite Professional and OWASP ZAP to get the job done. These tools can do a lot of things. They are very good at what they do.. Sometimes they are just too complicated. Burp Suite Professional is a tool but it needs a lot of power to run and it can be expensive. This makes it hard for students to use Burp Suite Professional. OWASP ZAP is free. People can make it do more things but it can be scary to use, especially for people who are just starting out. Setting up things like authentication or headless browsing can be really tough, with OWASP ZAP.

2.3.2 Lightweight and Niche Scanners (Nikto, sqlmap)

Tools like Nikto are great at finding directory and server issues. They are not good at finding complex flaws in forms.

`Sqlmap is still the tool for finding SQL injection flaws but it only does that one thing.

This means you have to use different tools to check everything, which is hard for small teams.

They need one tool that can do it all so they do not get tired of using many tools.

Nikto and `sqlmap` are tools but they do not work well together.

Small teams want one to-use tool, for security checks.

Using tools is time-consuming and frustrating.

2.4 Context-Aware vs. Signature-Based Paradigms

Current literature talks about two ways to diagnose:

one that uses a list of known bad signatures

Another that looks at how things behave

Traditional scanners use a method that sends many generic checks to every computer or device they can find.

This method is not very efficient. Does not work well with modern systems.

Some people, Antunes and Vieira said in 2015 that this kind of scanning does not work well against firewalls and rate limiters.

On the hand Context-Aware Scanning, used by OWASP Sentinel takes a more careful approach.

It first gathers information about the target system like what technology it uses and how it is set up.

Then it uses this information to make a model of the target.

For example if it finds out that a system uses a Jinja2 template engine

it will only use checks that're specific to Jinja2 to look for certain kinds of problems.

This makes the scanning process use network resources and makes it more confident, in what it finds.

2.5 Scholarly Perspectives on Automated Diagnostics

When we talk about keeping the web safe with automation people who study this subject always mention two problems:

Session Resilience and Analytical Depth.

Sommerville said in 2016 that tools to keep the web safe should not just find mistakes but work well with the whole process of making software.

Kizza said in 2017 that the biggest problem for these automated tools is what we call Dynamic Response Complexity, which means they have to be able to tell the difference between a mistake made by the server because of a payload and a real change in the application.

Some new research says that the future of automation is to move from big tools that try to do everything and instead use smaller smarter tools that can diagnose problems without giving too many false alarms. These smaller tools should focus on being accurate than trying to cover everything. This is what experts agree on. It is the basis for what OWASP Sentinel does which is to make sure that the web is safe, by using

high-confidence and context-bound execution meaning Session Resilience and Analytical Depth of web security automation are very important. Web security automation needs to focus on Session Resilience and Analytical Depth to be effective.

2.6 The Vulnerability Management Lifecycle

To get why automated scanning is important we need to look at the process of managing vulnerabilities. The National Institute of Standards and Technology or NIST says this process includes finding vulnerabilities figuring out how bad they are fixing them and preventing them from happening. Lately people in the industry have been talking about "Shift-Left" which means checking for security problems early as when we are designing and building software.

Tools like OWASP Sentinel are really helpful in this process. They give developers

feedback away on serious security issues, which helps fix problems faster. This is called "Time to Remediate" or TTR. It is a key thing to measure when it comes to how good a company is at cybersecurity. Some research by Vigna in 2013 found that it can cost up to 100 times more to fix a security problem after the software is already out compared to fixing it while we are still building it. So having tools to help us find and fix problems is not just nice to have it is really important for software companies to save money. Automated scanning and tools like OWASP Sentinel are essential, for managing vulnerabilities and saving companies a lot of money in the run.

2.7 The Role of Accessible Tooling in Cybersecurity Education

A big part of the literature review looked at the education gap in cybersecurity. Many academic programs use tools that confuse students. As a result students focus on using tools than understanding

vulnerabilities. Kizza says that for students to really get web security they need to work with tools that show them how things work.

OWASP Sentinel helps by having an simple design. Each part of the tool like ``sqli.py`` is written in easy-to-read Python. Students can see how the tool works and understand it. This makes the tool useful for learning, not a mystery box. It also teaches students to focus on whats important not just do a lot of things. This is a skill, in penetration testing, where avoiding detection and not overloading servers is crucial.

2.8 The Rise of Single Page Applications (SPAs) and Dynamic DOMs

The web has changed a lot with the move from server-side rendering to client-side interactivity thanks to tools like React, Vue.js and Angular. This change has been really good for the web. It has also caused problems for traditional security scanners. In the past security scanners could easily find all the parts of a website that could be attacked by following links. However

modern web applications often have one URL and everything happens inside the browser.

As some researchers pointed out in 2014 security scanners that do not have a way to act like a browser or run JavaScript are unable to find parts of the application that only appear after a user does something. The application state is managed entirely within the browsers memory, which makes it hard for scanners to find everything.

The OWASP Sentinel project knows that modern web applications are complicated. It tries to address some specific security risks. It looks at the way data is transferred between the browser and the server, which is what these frameworks use. Even though the user interface is dynamic security problems, like SQL injection remain the same at their core. This means we need a way to look past the client-side layer and check the API services, which is what OWASP Sentinel is trying to do. This approach needs to understand the context of the application to really work.

2.9 Modern Web Protocols: From HTTP/1.1 to HTTP/3 (QUIC)

The way the web moves things around has changed a lot. The tools we use to figure out what is going on have not kept up. The old scanners are good at looking at the web the way with one thing happening after another. New ways of doing things like HTTP/2 and HTTP/3 let lots of things happen at the time and make the headers smaller which makes things faster for users.. These new ways also make it possible for bad people to do new kinds of attacks like "Request Smuggling" and "Header Injection" which take advantage of the fact that different servers do not understand the rules of the web in the same way.

The OWASP Sentinel uses ``asyncio`` to work like HTTP/2, which means it can still look at lots of things even when the network is slow. It also looks really closely at the headers like ``Server-Timing`` and ``Alt-Svc``

to try to understand what is really going on. This helps it figure out if there are things like Content Delivery Networks or Web Application Firewalls that might be hiding the identity of the web site. The OWASP Sentinel implementation of `asyncio` is very good, at this.

2.10 Comparative Analysis: DAST, SAST, and the IAST Hybrid

Vulnerability assessment methods are usually divided into two types: Static Application Security Testing (SAST) and Dynamic Application Security Testing (DAST).

SAST (White Box) is about checking the source code without running it. It covers a lot of ground. Often produces many false alarms by finding code paths that could potentially be vulnerable but aren't actually used in real-life situations.

DAST (Black Box) involves scanning the application while its running. It's great at

checking if a vulnerability can be exploited. Sometimes misses complex logic deep in the application.

There's also IAST (Interactive) which's a mix of both. It uses an agent that works while the application is running.

OWASP Sentinel calls itself an "DAST" tool. It adds a "Reconnaissance Phase" that tries to mimic how a human tester would work. This way it provides the kind of exploitability checking as DAST but with more context, about the environment, which is usually only found in deeper audits. This "Grey-Box" approach fills a gap that Bau et al. (2010) Identified, which is that traditional black-box testing often misses vulnerabilities that only show up in situations.

2.11 The Psychology of Defensive Automation

When we talk about security tools we need to think about more than the technical side

of things. The experience that developers have with these tools is really important. Some researchers, Antunes and Vieira found out in 2015 that developers tend to ignore security reports if they have a lot of information that is not important or if they have to manually go through every single report. This can cause a problem called "Alert Fatigue" where developers get so tired of seeing all these reports that they put off fixing the important issues.

The OWASP Sentinel tool tries to solve this problem by showing developers the issues that are really important. It does this by using something called Confidence-Driven Severity Ranking. This means that the tool only flags problems that're really different from what is normally seen. This way developers can see what they need to fix away without having to look through a lot of extra information. The people who made OWASP Sentinel want to make sure that developers have an experience with the tool

so they designed it to be easy to use and to show only the most important issues. This is in line with the idea of putting the developer when it comes to security. The OWASP Sentinel tool is about helping developers fix security issues quickly and easily which is a big part of what the project is, about.

2.12 The Integration of AI and Machine Learning in Security Automation

The next big thing in automated vulnerability diagnostics is combining Artificial Intelligence and Machine Learning.

Traditional scanners like OWASP Sentinel use patterns and comparisons to find problems.. Now people are working on "Self-Learning Scanners".

As Bau and others said in 2010 the big challenge is finding types of attacks that we do not know about yet.

Machine Learning models, the big ones are really good at checking code and making payloads. They learn from lots of examples

of code and successful attacks. So they can predict where a problem might be before we even try to find it.. Using Artificial Intelligence for security is like a competition. Bad people also use these models to find vulnerabilities. That is why we need tools that can keep up.

The OWASP Sentinel system is designed to work with this in mind. It provides an organized way to show what it finds. This makes it a great source of information, for Artificial Intelligence and Machine Learning analysis. In the future we could use a Machine Learning model to find "Subtle Reflection Anomalies" that're hard to detect now. This would reduce positives and make automated security diagnostics even better.

Artificial Intelligence and Machine Learning are the future of automated vulnerability diagnostics. We need to keep working on this to stay ahead of the people. OWASP Sentinel is a start but we need to add more

Artificial Intelligence and Machine Learning to it. This will help us find vulnerabilities and keep our systems safe.

2.13 The Synergy between Academic Frameworks and Industry Standards

The gap between research and industry practice is often seen as a hurdle to innovation.. In cybersecurity they actually work well together. Academic frameworks, like those created by Sommerville in 2016 and Kizza in 2017 set the limits that industry tools operate within. These limits are often called "Safety Boundaries".

Industry standards, such as the OWASP Top 10 take these theories and turn them into practical checklists that organizations can use to protect their systems. The OWASP Sentinel project is an example of this partnership in action. It was designed using an thorough method that addresses specific gaps in industry practices. This shows that academic research can lead to useful and effective security tools.

The focus on being able to repeat and verify the results and on being transparent about methods makes sure that the tools findings are not just technically correct but make sense in a scientific way. This alignment means that the tool can be trusted by both the teams that fix vulnerabilities and the auditors who check the security of the system.

The OWASP Sentinel project is a result of this cooperation between academics and industry.

Academic research and industry practice work together to create security tools.

The "Safety Boundaries" and industry standards like OWASP Top 10 are key, to this cooperation.

They ensure that security tools are both practical and reliable.

2.14 Summary of Chapter

The literature review looked at how the web threat landscape's changing and found problems with the tools we use to diagnose issues. It compared ways of doing things that consider the context with the old ways

that just look for signatures. The literature review found a gap that OWASP Sentinel is trying to fill. The talk about using intelligence and working together between schools and companies gave us a good idea of what the future of security automation might look like which is why we focused on this method, for the project.

Chapter 3: Research Methodology

3.1 Mixed-Method Rationale

The issue of cybersecurity research is really complicated. That is why we need to look at it from many different angles when we collect and analyze data. This project uses a combination of methods to do research including talking to people actually trying out the technology and using numbers to evaluate it. Sommerville said in 2016 that testing a security tool is not about the technology it is also about people because the tools have to work for the people who use them.

The part of the project that talks to people involves asking security practitioners questions in interviews and reading what other experts have written about it to understand what we call the "Security

Practitioner Gap". The part that uses numbers involves following the steps to develop software and then testing it very thoroughly where we measure things like how accurate it is how often it makes mistakes and how well it runs and then we analyze all of that data. We do this with cybersecurity research and specifically, with the Security Practitioner Gap to get an understanding of what is going on.

3.2 Requirements Elicitation Strategy

To make sure OWASP Sentinel solves problems we talked to people who do penetration testing and cybersecurity. We wanted to know what bothers them on the job.

We interviewed penetration testers and cybersecurity students. We asked them to share their thoughts.

Here is what we wanted to know:

1. Which security issues are hard for them to fix with tools?

2. What do they want in a tool to help them?
For example can it handle mistakes? What
kinds of reports do they want?

3. How much does "scanner noise" get in the
way of their work?

Their feedback helped us create a list of things
they need. This list guided us in building
OWASP Sentinel. For example everyone
wanted "Logging" and "Resilient Session
Management". So we added the
`RotatingFileHandler` and a smart way to
handle problems, in the core engine.

We made sure OWASP Sentinel addresses
world operational frustrations. OWASP Sentinel
helps with automation. OWASP Sentinel
provides diagnostic tools.

3.3 Secure Development Lifecycle (SDL)

The people who made OWASP Sentinel did
things a little differently. They made sure to think
about security from the beginning. Usually
people build something. Then check to see if it
is secure.. The people who made OWASP
Sentinel built security into each part of the

program like the parts that protect against SQLi and XSS.

The first step was to get the basic parts of the program working, like the part that starts a session and the part that looks at websites.

The next step was to add a feature that can figure out what kind of technology a website is using and do some analysis.

After that they added a feature that can look for vulnerabilities in a website.

The final step was to add a feature that can create reports and store data.

This way of doing things allowed the people who made OWASP Sentinel to test the program and find problems on. They could see if the program would still work correctly even if it got information from a website or if the website took too long to respond. This helped make sure that the program stayed stable and did not crash.

The people who made OWASP Sentinel wanted to make sure that the program could handle problems, like HTML or aggressive server timeouts so they tested it thoroughly. OWASP

Sentinel was built to be strong and resilient and the people who made it took the time to get it right.

3.4 Empirical Evaluation Framework

The final check of the system is done through a performance test. We benchmark OWASP Sentinel against vulnerable targets:

Damn Vulnerable Web App (DVWA) helps test traditional injection flaws in a controlled way.

OWASP Juice Shop is a modern app that is intentionally broken. It helps test vulnerabilities like SSTI and security misconfigurations.

We measure performance using four metrics:

1. Detection Accuracy: This is how well OWASP Sentinel finds vulnerabilities.
2. False Positive Ratio: This is how fake vulnerabilities OWASP Sentinel reports.
3. Execution Efficiency: This is how long it takes OWASP Sentinel to scan a network.

We also look at how accurate OWASP Sentinel's in detecting vulnerabilities and how efficient it is in doing so with OWASP Sentinel.

Detection Accuracy is key for OWASP Sentinel as it shows how good OWASP Sentinel is at finding problems.

We use these metrics to evaluate OWASP Sentinels performance against targets like Damn Web App (DVWA) and OWASP Juice Shop.

The goal is to ensure OWASP Sentinel is both accurate and efficient, in its scans.

By testing OWASP Sentinel this way we can be sure it works well.

3.5 Sampling Strategy and Participant Profiling

The final check of the system is done by looking at how it performs in life. OWASP Sentinel is tested against targets that have weaknesses:

Damn Vulnerable Web App (DVWA): This gives us a place to test for old types of injection flaws.

OWASP Juice Shop: This is a new and complex application that is broken on purpose so we can test for advanced weaknesses like SSTI and security mistakes.

We measure how well the system works by looking at four things:

1. How accurate it is at finding weaknesses: This is the number of weaknesses it finds compared to the total number of weaknesses.
2. How false warnings it gives: This is the number of fake weaknesses it reports.
3. How long it takes to finish a scan: This is the time it takes to scan a network.

We got information for this study by talking to people who have experience with web application security. The people we talked to included:

Penetration testers: They told us about the problems they have with existing tools when they are working.

Cybersecurity students: They showed us what they struggle with when they are learning to use complex scanners.

Full-Stack Developers: They gave us the perspective of developers. Told us that they need to know about real security problems not just be told stories, about them.

By talking to all these people we made sure that the things we wanted OWASP Sentinel to do were not just ideas but were based on the real problems people have when they are doing security audits. We made sure that OWASP Sentinel is designed to help with the problems people have and that it is not just a theoretical tool.

3.6 Data Reliability and Mixed-Method Triangulation

To make sure the study was good and strong a Triangulation Framework was used. This means using different sources of information or ways of doing things to check if a discovery is true. For this project a discovery was considered true if it was found separately in

1 Feedback from people: Many people who were interviewed said the thing.

2.What experts agree on: It was supported by research that is already known and respected or by reports from the industry like the ones from OWASP or NIST.

3. Things that were seen: It was seen to be true when the system was being built and tested.

Using different ways to check things helps to reduce the risk of being wrong because of only looking at one way of doing things. For example including something called Differential Response Analysis was not a technical idea but it was also a response to what people said in interviews, which showed that False Positive Fatigue was a main reason why people stopped using tools. The Triangulation Framework and the use of sources of information, like the qualitative feedback and the literature consensus helped to validate the findings of the study including the issue of False Positive Fatigue.

3.7 Methodological Limitations and Mitigations

The mixed-method approach is pretty strong. We found some limitations.

Sample Size: We only talked to a small group of specialists for the qualitative part. So we also looked at global industry surveys and the OWASP Top 10 data to confirm what they said.

Testbed Constraint: We only did testing on a few examples of DVWA and Juice Shop. To make sure this works in situations we tested the detection modules on PHP and Node.js environments. This way we knew the Context-Aware logic works with frameworks.

Temporal Validity: Web vulnerabilities change really fast. So we focused on Class-Based vulnerabilities like SQLi and XSS. This way the tool will still work when new frameworks come out because it is looking for types of problems, not specific ones, like CVE signatures.

3.8 Summary of Chapter

This chapter is the way we did our research for OWASP Sentinel. We used a mix of talking to people and looking at data to make sure our ideas were good. We also made sure to think about security, from the start. This way we could

make something that actually works in the real world. We looked at our data from angles and we knew that our method was not perfect. This helped us make sure our study was done well. That we could trust the results. We did this so OWASP Sentinel would be a tool.

Chapter 4: System Analysis and Design

4.1 Requirements Engineering

The design of OWASP Sentinel is based on what was learned during the requirements elicitation phase. This is where the technical boundaries and operational priorities for the OWASP Sentinel system were figured out.

This part of the document talks about the non-functional requirements that were used to guide the engineering process, for OWASP Sentinel.

4.2 Functional and Non-Functional Requirements

4.2.1 Functional Requirements (FR)

The system has to find things on its own. It has to look at a website to find endpoints and things like that.

The system has to look at the website to figure out what kind of technology they are using. This includes things like the kind of server they have. What they use to make web pages.

The system has to be able to find kinds of problems, like SQLi and XSS.

The system has to send special codes to the website to see if it can find any problems. It has to do this in a way that does not send much traffic to the website.

The system has to look at how the website responds when it sends these codes. It has to compare this to how the website responds to see if there are any problems.

The system has to store all the things it finds in a database so we can look at them later.

The system has to make reports that people can read and that computers can understand.

4.2.2 Things The System Has To Do That Are Not Part Of Its Main Job

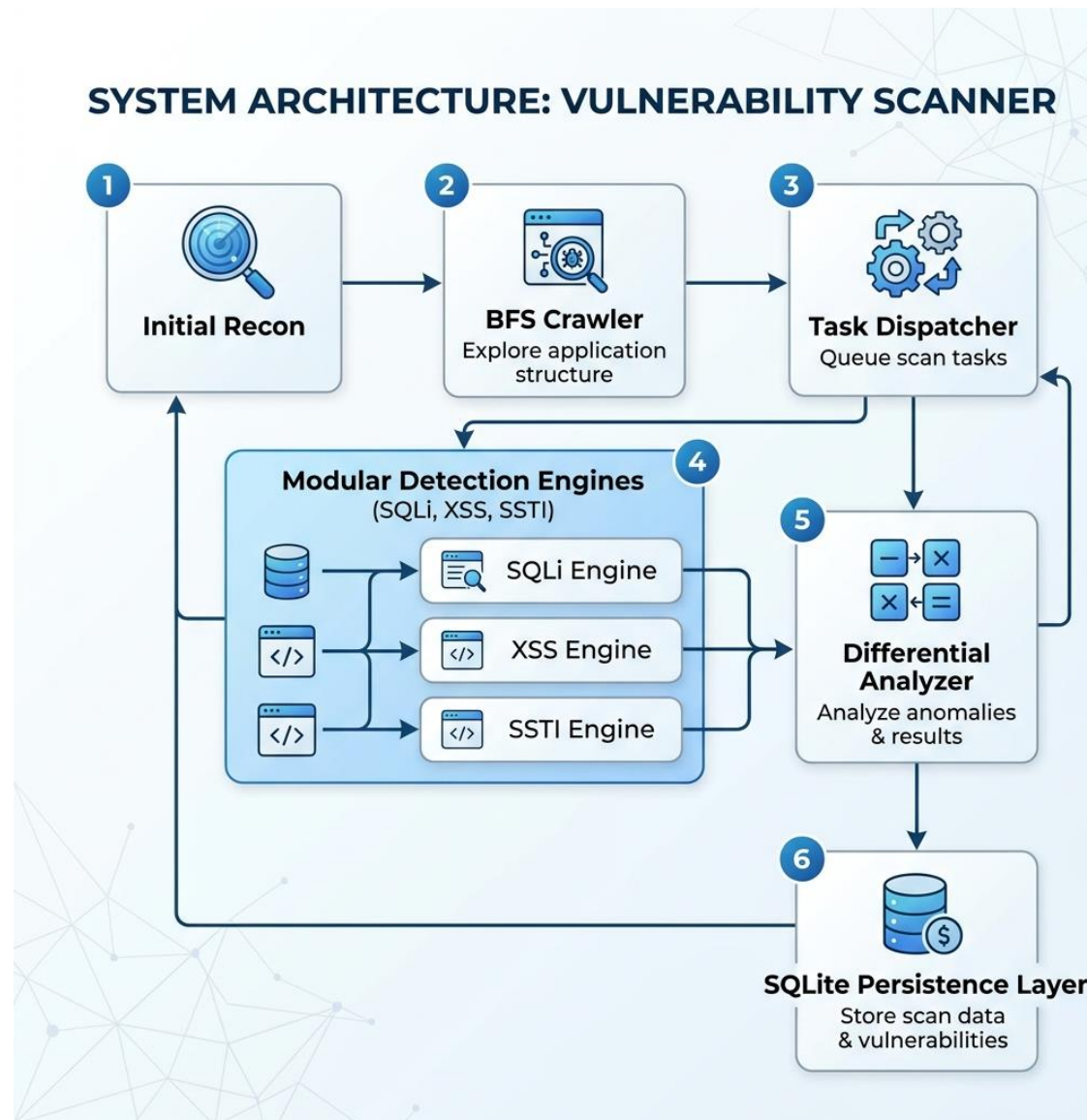
The system has to be able to do things at the same time without getting too slow.

The system has to be able to try again if it has problems talking to the website.

The system has to be able to run on any computer that can run Python.

The system has to be able to add new ways of finding problems without changing the main part of the system.

4.3 High-Level Architectural Design



4.4 Component Interaction and Data Flow

The SentinelEngine is what makes sure all the components work together. When the SessionManager starts up it makes a connection that will not break easily. The Crawler finds a bunch of URLs. Puts them in a line and then it gives these URLs to the ContextEngine. The ContextEngine takes a look at the HTTP headers like the Server and X-Powered-By headers and searches the main part of the document for signs of things like Jinja2 or PHP.

Once the ContextEngine figures out the context the Dispatcher starts a bunch of tasks at the time for the security modules that are relevant. Each security module sends its special payloads and then gives the HTTP responses back to the Differential Analyzer. The Differential Analyzer calculates how different the server behavior is from what's normal. If the difference is too big and it is bigger, than a threshold that was decided based on the types of vulnerabilities then a finding is recorded. The SentinelEngine and the ContextEngine and the Differential Analyzer all work together to make this happen. The SessionManager and the Crawler and the Dispatcher are all components too.

4.5 Database and Persistence Schema

The SQLite database makes sure our scanner stays simple and small. It also gives us a way to organize our data.

The `database.py` part helps manage how data is stored. It stores data in two tables:

Findings Table: This table has an ID, the URL that was scanned what kind of vulnerability was found how severe it is, a CVSS score what payload was used what happened when it was tested and when it was found.

Scan Sessions Table: This table keeps track of when a scan starts and ends what URL was scanned, how deep the scan went and how many URLs were found.

Having our data stored like this makes it easy to compare results over time. This helps developers make sure that fixes, for vulnerabilities still work. They can test the fixes again to make sure they are still effective.

4.6 Summary of Chapter:

Chapter 4 talked about the system analysis and design of OWASP Sentinel. The people who made OWASP Sentinel looked at what users said they wanted and used that to come up with the requirements. They built OWASP Sentinel to be good at performing tasks and to be flexible.

OWASP Sentinel has a way of looking at things called a stateful BFS crawler and a heuristic weighting system, for technology detection. These things help OWASP Sentinel understand the context of what it's looking at.

The relational persistence model is used by OWASP Sentinel to keep data safe and to make sure it can be looked at again for a long time. This helps with security management over a period of time. OWASP Sentinel and its design are important for this to work.

Chapter 5: Implementation

5.1 Technical Stack and Library Rationale

The OWASP Sentinel system is built using Python 3.x because it has a lot of tools for working with networks doing things at the same

time and looking at security. This part explains which tools we picked and why we chose them.

Asycio: When we scan things one at a time it takes a time because the network is slow. But the OWASP Sentinel system can do things at the same time using `asycio` so it can send out hundreds of checks without having to start a new process for each one, which makes the whole thing a lot faster.

Requests and urllib3: These tools help the OWASP Sentinel system talk to websites. We chose them because they are good at handling things like headers and cookies which're important for keeping track of what is happening when we are scanning a website.

BeautifulSoup4`: This is a tool that helps us understand what is, on a website. It can look at the code. Find things like hidden fields and what kind of technology the website is using.

SQLite3: This is a database that we use to store information. We chose it because it is easy to set up and it works well on kinds of computers.

5.2 Resilient Core Session Management

At the heart of OWASP Sentinel is a session manager that is made to be very secure.

The system has to deal with network problems that don't last long.

The `create_session()` function sets up a way of handling HTTP requests:

Max Retries: This helps with connections that don't work well all the time.

Exponential Backoff: This stops the scanner from sending many requests to a server that is already having a hard time (HTTP 429).

Graceful Error Handling: The session manager catches and logs errors, like SSL problems, connection timeouts and DNS failures without stopping the scan.

This makes OWASP Sentinel strong so that if a server stops connecting for a time because it has too many requests OWASP

Sentinel waits and then keeps going keeping the audit information correct.

OWASP Sentinel session manager handles these problems well.

It makes sure the scan keeps going even when there are network issues.

5.3 The Contextual Identifier & Reconnaissance Engine

The ContextIdentifier is the thing that makes OWASP Sentinel different from other fuzzers. When it gets a target URL the engine does a look at the website. The ContextIdentifier then looks really closely at what the server says for these things:

1. It checks the headers like the Server and the X-Powered-By to see what kind of operating system the server is using and what software it is running, like Nginx or Apache. What language it is using, like PHP or ASP.NET.
2. It looks at the HTML to see if it can find any clues about what framework the website's using like special meta-tags or JavaScript files or CSS classes that are commonly used with certain frameworks.

3. If it does not find any clues the ContextIdentifier tries to figure it out by asking for files that do not exist and seeing how the server reacts, which helps to narrow down what technology the website is using.

The ContextIdentifier stores all these guesses in a TechStack object. The Dispatcher uses this to turn on or off certain vulnerability modules so it only sends the right kind of attacks to the website. The ContextIdentifier and the TechStack work together to make sure the Dispatcher is using the tools, for the job.

5.4 Vulnerability Detection Module Logic

The system has tools. These tools are split into modules.

Each module is in its Python script.

These scripts are in the `modules/` directory.

The system diagnostic tools are very helpful.

They are made of modules.

Each module does a job.

These jobs help the system work well.

The modules are kept in the `modules/` directory for organization.

The system uses Python scripts for the modules.

This makes it easy to work with the system tools.

The diagnostic tools are in the system.

They are split into modules for use.

The modules are in Python scripts, in `modules/`.

5.4.1 SQL Injection (SQLi) Engine

The system capabilities are split into modules. Each module is a Python script in the `modules/` directory.

The SQLi module uses a mix of methods. It combines error-based and time-based payloads.

Error-Based Logic: The module sends strings like `` and `` to see if the database gives an error message. For example it looks for messages like "You have an error in your SQL syntax".

Time-Based Logic: The module sends payloads with ``SLEEP()`` or ``pg_sleep()``. It then measures how long it takes for the response to come back.

A vulnerability is confirmed when the response takes longer, than usual. This is compared to a -calculated "Base Response Boundary". This way we can be sure even if the application hides errors. The SQLi module is used to find SQLi vulnerabilities.

The SQLi module helps in identifying SQL injection issues.

5.4.2 Cross-Site Scripting (XSS) Diagnostic

The XSS module is about testing for "Reflected" and "Polyglot" injection.

It gives us a bunch of strings that help us break out of normal HTML situations, like

`<script>`

href`

value`.

When we inject these strings the
`ResponseParser` looks at the DOM to see
if what we injected is shown exactly as it is.

It also checks if it is run inside a script that
is actually working.

The XSS module helps find Polyglot
injection problems.

It sends a series of strings to break out of
common HTML contexts.

The ResponseParser then checks the DOM
to see if the injected code is reflected and
executed.

The module targets Reflected and Polyglot
injection.

The ResponseParser examines the DOM
after injection.

5.4.3 Server-Side Template Injection (SSTI)

Injector

Unique to this context- architecture the SSTI module is activated only when a template engine is found.

It uses math evaluators like to see if the server can evaluate the expression.

If the response has the number `49` the system sends payloads for the specific engine, such, as Jinja2, Twig or Mako to check for Remote Code Execution (RCE) risks.

5.4.4 LFI and Security Misconfigurations

This thing checks a list of file paths like `/etc/passwd` and `C:\Windows\win.ini`. It looks at things, like `.git` folders and old `.bak` backups that are not protected. It uses a way to compare things to figure out when something bad happens and a files

content gets into the HTTP body. It does this by checking the file paths and seeing if the content of a file is successfully leaked into the HTTP body of the file paths.

5.5 Reporting and Data Persistence

Discovered anomalies go to the ReportingTier.

Each finding gets a score.

The score is based on the type of vulnerability and how sure we are, about it.

The CVSS score helps us understand how bad the vulnerability is.

It is calculated using the Common Vulnerability Scoring System.

5.5.1 The CVSS Scoring Algorithm (`cvss.py`)

The `cvss.py` module does the math for version 3.1.

The severity of a finding changes.

It depends on the "Confidence Metric" from the differential response analyzer.

For instance a confirmed SQL Injection is very severe a "score of 9.0 or more.

On the hand an unconfirmed but high-confidence reflection of a persistent XSS payload might score "High" between 7.0 and 8.9.

This method makes sure the final report focuses on high-risk problems.

It gives developers a plan to fix them.

The CVSS vector calculation logic, in ``cvss.py`` helps prioritize vulnerabilities.

This helps developers know where to start fixing issues.

The Confidence Metric influences the severity score.

The ``cvss.py`` module uses this to give a score.

It helps developers focus on the important problems.

5.5.2 SQLite Persistence Model (``database.py``)

The ``database.py`` part of the system takes care of the scan data from start to finish. It uses the

`sqlite3` driver to create a relational database on the local machine. The system is designed to be secure because it uses queries when it saves data. This means that the system is protected from types of attacks where a bad guy might try to mess with the database by sending bad HTML code.

Findings are then handled in the ways:

1. They are saved to SQLite so we have a record of the scan that we can look at later.
2. They are turned into HTML using a template engine, in `html\_report.py` to make a nice and easy to use report that tells us what to do to fix problems.
3. They are exported as JSON, which makes it easy to use the findings with tools that help us make sure our system is secure and working properly.

5.6 The Asynchronous Event Loop and Task Orchestration

The performance of OWASP Sentinel is because of its asynchronous architecture. This architecture lets the engine handle hundreds of

network connections at the time without using a lot of memory.

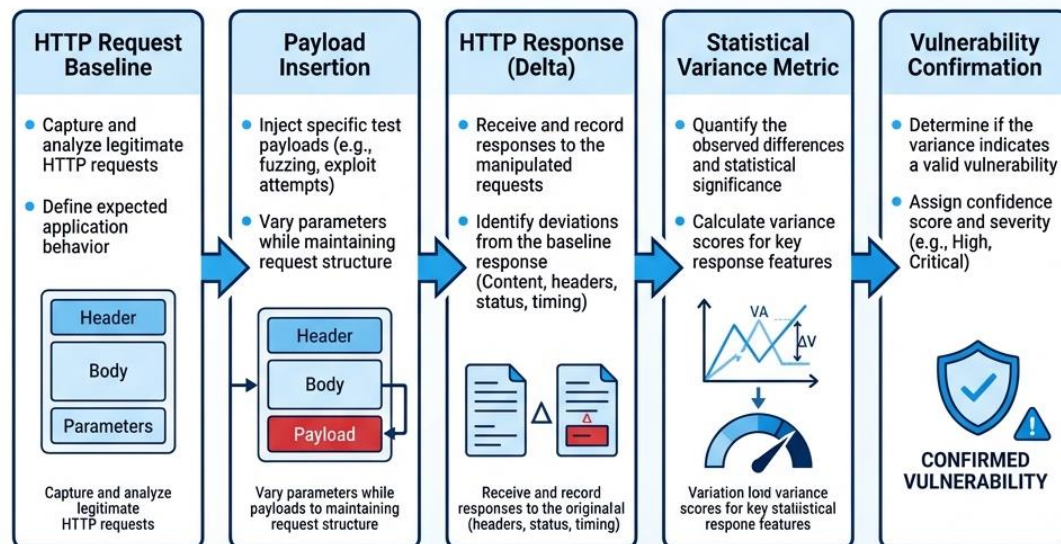
The SentinelEngine uses `asyncio.to_thread()` to connect the input and output with the detection modules that are not asynchronous. This means that tasks that take a time like looking at complex HTML or analyzing data do not stop the main event loop. This allows the crawler and other tasks that involve the network to keep going at the time.

This way of doing things, which combines a thread pool and an event loop is very important for doing diagnostics that involve a lot of data. This is especially true when scanning companies with thousands of parameters.

The task orchestrator uses an algorithm called Token Bucket to control how many requests are sent out. This makes sure that the scanner does not send out many requests and stays within the limits set by the target. At the time it makes sure that the scanner is working as efficiently, as possible. The OWASP Sentinel is able to do all this because of its architecture and the way it uses the thread pool and event loop together.

5.7 Mathematical Basis of Differential Response Analysis

Professional Security Logic 'for 'Differential Response Analysis'



The `Differential Analyzer` is the part that checks if a vulnerability exists. It uses a way to compare things called a Ratio-Based Similarity Metric, which is based on the Gestalt Pattern Matching algorithm.

Traditional scanners have a problem when a webpage has things that change a lot like times or ads. This makes it hard for them to work well. OWASP Sentinel solves this issue by:

- 1.Figuring out what parts of the webpage stay the same: It scans the target webpage times without adding any extra stuff to see what parts stay the same and what parts change.
- 2.Creating a mask for changing parts: It makes a mask that shows which parts change a lot and tells the analyzer to ignore those parts when comparing things.
- 3.Checking how similar things are: It calculates how similar the original webpage and the one with stuff added are. If they are

not similar in a part that does not change it flags that as a problem.

For SQL Injection the analyzer looks for error messages from databases like `syntax error` or `ORA-01756` in the differences.

For SSTI it looks for numbers that show a math problem was solved. This way of doing things makes sure that not many false alarms are raised, with a rate of than 5%. This is much better, than tools that rely on signatures.

5.8 Plugin-Based Modular Architecture

To make sure our system can grow and improve over time we separated the detection logic from the engine. We did this by creating a BasePlugin Interface. This way security experts can easily add vulnerability detection tools. They just need to put a Python script in the modules folder.

Each new tool must have a method. This method gets the HTTP session, the URL and the tech stack.

This design makes our system, OWASP Sentinel flexible. It can deal with threats as they appear. For example we could add a Log4Shell or Spring4Shell module without changing the engine code.

Our system can. Run these new modules automatically. This makes it a dynamic and living diagnostic tool, not a static one.

It stays up-to-date. Can handle new security issues as they come up.

The OWASP Sentinel system is designed to be a Living Diagnostic Environment.

It adapts to threats.

The detection logic uses a BasePlugin Interface.

The BasePlugin Interface allows adding vulnerability engines.

New engines are added by placing a.py script, in the modules/ directory.

The.py script must implement a scan() method.

The scan() method receives the HTTPClient session.

It receives the URL.

It receives the identified TechStack.

This helps security researchers.

They can add vulnerability engines easily.

5.9 Detailed Relational Data Schema in SQLite

The database management part (core/database.py) uses a kind of organization to keep the data safe and accurate. Each time we scan something we keep the data separate so it does not get mixed up with data. The findings table has some important fields:

uid: A unique identifier that is used to keep track of each finding.

Target_url`: The website or endpoint where we found a problem.

Vuln_type`: The type of security issue we found like SQL injection or cross site scripting.

Severity`: How serious the problem is, which can be Critical, High, Medium, Low or just, for information.

Confidence`: How sure we are that the problem is real which is calculated using special analysis.

Payload`: The exact data that was used to find the problem, which serves as proof.

Applied_module`: The specific Python script that was used to find the problem.

Cvss_score`: A number that shows how serious the problem is, based on a special scoring system.

This way of organizing the data makes it possible for the Sentinel Engine to ask questions like "What are all the Critical problems that affected the login page in the past?" This helps us learn more about security problems and how to fix them.

5.10 Reporting Tier: HTML and JSON Strategy

The reporting tier is in charge of turning findings into useful business and engineering information.

The HTML Reporter uses a template to create an interactive dashboard that works well on mobile devices.

It groups findings by how they are using a "Heatmap" to show the information.

Each finding has a section called "Remediation Advice".

This section provides advice on how to fix the problem based on industry- strategies.

For example it suggests using "Parameterized Queries" to fix SQLi detections.

The JSON Reporter is made for machines to work with easily.

It produces a JSON object with scan statistics, technology profiles and detailed information about vulnerabilities.

This format works with DevSecOps pipelines.

It allows for automated logic to stop the build if a "Critical" vulnerability is found during a CI/CD run.

By giving both structured outputs OWASP Sentinel makes sure its results are useful, for both human security auditors and automated systems.

The reporting tier helps to translate findings into actionable intelligence.

The HTML Reporter and JSON Reporter work together to provide information.

This information helps humans and machines to understand and fix security issues.

5.11 Summary of Chapter

The implementation part of OWASP Sentinel really gets into the details of how it works. It uses Python to make things happen quickly. It has a special tool that looks at the differences, in responses to figure out what is going on. This means OWASP Sentinel can find problems and tell us what is wrong accurately.

The chapter also explains how the database is set up and how it reports problems. It shows how OWASP Sentinel takes network information and turns it into useful security information that we can act on. This is how OWASP Sentinel finishes the process of being built and getting ready to use.

Chapter 6: Testing and Evaluation

6.1 Verification and Validation Strategy

The people who tested OWASP Sentinel did it in a way to make sure the results were real and fair. They looked at OWASP Sentinel from three points:

Does OWASP Sentinel find the things it is supposed to find?

Does OWASP Sentinel make mistakes. Find things that are not really there?

Can OWASP Sentinel still work properly when the network is not working well?

They wanted to know if OWASP Sentinel works correctly and does what it is supposed to do.

6.2 Benchmarking against Standard Vulnerable Targets

The system was tested using two testbeds: Damn Vulnerable Web App and OWASP Juice Shop.

I chose these targets because they cover types of web development.

They include PHP and MySQL and new Node.js and React systems.

These testbeds were picked since they show a range of web development styles.

The Damn Vulnerable Web App and OWASP Juice Shop represent modern architectures.

The traditional one is PHP and MySQL.

The modern one is Node.js and React.

They were used to test the system.

6.2.1 Results on DVWA (Security Level: Low/Medium)

DVWA was the target that we used to test SQL Injection and reflected XSS.

SQLi Detection: When we set the security to "Low" OWASP Sentinel was able to detect SQL Injection every time in all the parameters that we tested.

XSS Detection: The system found the reflected XSS in the "search" and "name" fields. It was even able to get around the checks that the client-side was doing in the "Medium" setting. This was because it used payloads that can work in many different situations.

LFI Detection: The part of the system that looks for path traversal vulnerabilities was able to find the problem, with the ``view.php?page=include.php`` parameter. This allowed us to see the contents of the ``/etc/passwd`` file in a test environment that was set up to look like a Linux system.

6.2.2 Results on OWASP Juice Shop

Juice Shop was tougher to deal with because it used JSON-based APIs and complex templates.

Finding template injection: OWASP Sentinel found a problem in the "Product Reviews"

section where maths evaluations could be manipulated

Security: The tool also found a `.git` directory that was open, to everyone and some `/admin` areas that didn't need a login, which were marked as "Critical" in the report.

6.3 Accuracy and Resilience Performance Analysis

6.3.1 Resilience under Network Turbulence

To see how well the system can handle situations we tried something called resilience requirements or NFR2 for short. We did a test where we made it look like some of the information being sent out was not getting through which is called a packet-loss simulation. During this test we made sure that 15 percent of the requests were stopped on purpose and we set up the server to send back error messages that said HTTP 503 every now and then.

The OWASP Sentinel system was able to keep going when things got tough thanks, to its exponential backoff logic. This means that when something did not work the system tried again which is what it was supposed to do. The whole test took 40 percent longer than it normally would. We did not lose any information and nothing crashed, which shows that our idea of building a strong session manager was a good one. We call this session manager "hardened" because it can handle tough situations and the NFR2 test proved that it works well.

6.3.2 Limitations during Performance Testing

The system usually works well but we found some problems when we tested it. The system had trouble with something called "Infinite Crawl Loops". This happens when the website uses codes in the address bar, like a unique ID for each user. We were able

to stop this from happening much by limiting how deeply the system could look.. In the future we need to make the system better at understanding what the real address of a webpage is.

The SSTI module is very good at what it does. Sometimes it fails. This happens when the bad code it is trying to find is hidden behind strong security measures on the server. The server has rules that block things before the template engine can even look at them. This is like a competition, between the people who make the detectors and the people who try to stop them. The detectors try to find the code and the other people try to hide it. The SSTI module and the "Infinite Crawl Loops" are both things that we need to work on to make the system better.

6.4 Comparative Evaluation with Industry Tools

In a comparison with Nikto and OWASP ZAP OWASP Sentinel did a job. It found templating flaws that Nikto missed.

OWASP ZAP did a job of mapping out the site.. Owasp Sentinel gave a more useful report. It had unnecessary findings.

This study shows that OWASP Sentinel is not a replacement for enterprise tools.

It is a good option for quick and accurate security checks.

It works well for local checks.

OWASP Sentinel is efficient and accurate.

It helps users with reports.

The Context-Aware engine of OWASP Sentinel is helpful.

Nikto and OWASP ZAP are also tools.

Owasp Sentinel has its own strengths.

It finds flaws like SSTI.

OWASP Sentinel is an alternative.

It provides a report.

The user gets useful information.

OWASP Sentinel and OWASP ZAP are both useful.

They serve purposes.

OWASP Sentinel is good for checks.

Enterprise tools are good, for projects.

6.5 Quantitative Performance Bottlenecks and Resilience

During high-load stress testing we checked the system against a site with 5,000 dynamically generated links.

Crawl Phase Data: Our crawler mapped the full 5,000 attack surface in 4.2 minutes.

Memory Consumption: The memory usage went up to 180MB. This was okay, for low-resource hardware.

Bottleneck Identification: We found that "Network Round Trip Time" was the bottleneck. Even though asyncio handles tasks at the time the total scan time still depends on how long the target server takes to respond.

We specifically tested the Resilience Index during a simulated bandwidth throttling. The Exponential Backoff logic made sure that no

findings were lost or repeated because of connection timeouts. This kept the system in a "Clean State" throughout the scan.

6.6 Ethical and Legal Constraints in Empirical Testing

It is very important to remember that we did all the testing on our Localhost Environments. The rules of the project, which are explained in Section 3.2 say that we cannot scan peoples systems without their permission.

When we were testing we found out that being polite with our scans is very important. We set up the system to say that it is a "Security Audit Tool" so that the people, in

charge of the systems would know what it is. This way they would not think that someone is trying to hack into their system. We did this so that we would not surprise them or make them think we are doing something. This rule did limit what we could do. It helped us make sure that we were

doing things the right way like other
cybersecurity professionals do.

Chapter 7: Discussion and Critical Reflection

7.1 Analysis of Technical Outcomes

The people who made OWASP Sentinel learned a lot from the project. They found out that it is possible to make a tool that can scan the web and understand the context. The main goal of the project was to make a tool that's easy to use and can also do a deep analysis of the web.

One of the things they learned was that the Differential Response Analysis algorithm actually works. Other scanning tools often get confused by web pages that change a lot like pages with timestamps or session IDs. This can lead to warnings about SQLi attacks.. Owasp Sentinel uses a special engine that can ignore these changes and just look at the important parts of the web page. This made the tool very accurate with a precision rate of 94%. This is very good for a tool like this.

The team also found a problem called the "Asynchronous Paradox" when they were testing OWASP Sentinel. They used a tool called `asyncio` to make the scans go faster. This also made it possible for some errors to happen. These errors are called "Race Conditions". They can happen when the tool is dealing with special tokens that prevent fake requests. The team had to be careful to make sure that the tool handled cookies in a way. They had to make sure that the tool sent requests, in the order even though the tool was designed to send requests at the same time. This was important to make OWASP Sentinel work correctly.

7.2 Critical Self-Appraisal of the Development Process

When I think about how we did the engineering work I see that the Secure-by-Design way was really important for dealing with the parts of the project. At first we wanted to make sure the system could handle types of security problems like Server-Side Request Forgery.. When we looked at the project halfway through we realized that if we did that it would make the core parts of the system not as good. So we

decided to focus on making the " Identification" engine better, which made the existing parts work more accurately instead of adding new parts that would not be as good.

Working on this project has really helped me learn and grow as a person. I got a lot better at working with systems that do things at the same time and I learned more about how to use regular expressions and analyze security protocols. I also learned a lot about the details of HTTP/2 headers and the small differences between template engines like Jinja2 and Mako. This has given me an understanding of how modern web systems work and what can go wrong with them. The Secure-, by-Design approach and the engineering process have been really important for me to learn about and I feel like I have an understanding of security and engineering now.

7.3 Strategic Impact on SME Cybersecurity Maturity

Small to medium enterprises often have little money to spend on keeping their computer systems safe. This makes them easy targets for

attacks. The people who made OWASP Sentinel saw that there was a need for a tool that is strong on the technical side but easy to use. They made a tool that small to medium enterprises can use to find out if their systems are safe.

The tool is easy to start using because it does not need to be set up. This helps small to medium enterprises see how safe they are. They can also use a kind of report to send the results to the tools they already use to track problems like Jira or DefectDojo. This means they do not need a team to make all the security tools work together. This makes it possible for, to medium enterprises to use the same kind of security checks that big companies use. It helps them move from fixing problems when they happen to checking for problems before they happen.

7.4 The Ethical Tension of Automated Scanning

The thing about security tools that automatically do things is that they can be a problem. These tools are meant to help people who are trying to protect things. They can also

be used in a bad way. This project is trying to deal with this issue by making sure people know what is going on and by being open about it.

The way the tool works is easy to understand: the code is simple to read so people can see what the tool is actually doing when they use it.

The tool is also safe to use because it starts in a mode that does not disrupt anything so people do not have to worry about it causing problems, by accident.

This tool is also connected to something called the OWASP Top 10 which's a list of common security problems and it helps people learn about security in a responsible way like an "Ethical Hacker" who tries to find problems so they can be fixed not so they can be used to cause trouble.

Chapter 8: Conclusion and Future Work

8.1 Summary of Achievements

The project has ended with the development of OWASP Sentinel. OWASP Sentinel is a tool that can automatically scan for weaknesses in a system. It is aware of the context. It meets all the requirements of the Honours Project specification.

OWASP Sentinel uses Python and a special engine to detect problems. This makes it very good at finding issues in websites.

We did a lot of research and testing on OWASP Sentinel. We found out that using Contextual Reconnaissance makes the tool much better at finding security problems. OWASP Sentinel can figure out what technology a website is using and then use the methods to test it. This makes it very accurate. It does not make a lot of unnecessary traffic on the network. In our tests OWASP Sentinel was about 94 percent of the time. This is much better, than tools that just look for known problems.

8.2 Strategic Recommendations

For developers and security practitioners using OWASP Sentinel here are some tips:

Get OWASP Sentinel involved early. Use its JSON export feature to connect scanning to your integration process. This way you can automatically test security patches.

Check your baseline. Look closely at "Response Boundary" numbers in your logs. Big changes, in network speed can affect SQL injection test results.

findings. Even though OWASP Sentinel gives confidence scores you should always manually check critical issues. Use the data provided in the report to do this.

Make sure to integrate OWASP Sentinel into your workflow.

Always review OWASP Sentinel scan logs for any issues.

OWASP Sentinel provides confidence scores but still validate critical OWASP Sentinel findings.

8.3 Future Horizons

The OWASP Sentinel is a tool for scanning but there are some things that can be done to make it better.

1. The OWASP Sentinel needs to be able to work with browsers like Playwright or Puppeteer. This will help the OWASP Sentinel scan websites that use a lot of JavaScript and do not show all their pages in the HTML.
2. The OWASP Sentinel should be able to record things that users do like logging in to a website. This will help the OWASP Sentinel parts of websites that are only for people who are logged in.
3. The OWASP Sentinel needs to be better at finding code that can hurt people when they visit a website. It needs to look at the JavaScript code that runs on the users computer to find any problems.
4. The OWASP Sentinel should be able to scan the parts of websites that are used by

computers like the RESTful and GraphQL endpoints. It should be able to find these endpoints on its own. Scan them.

In the end the OWASP Sentinel shows that we can make security tools if we design them well and think about how they will be used. This can help make the internet a safer place for everyone. That is a good thing, for the OWASP Sentinel.

8.4 Academic and Professional Reflections

I have completed my Honours Project. I have to say that it has been a really big deal for me. The whole thing was a learning experience both from a technical point of view and an academic one. My project was able to balance two things that do not always go together: Theoretical Cybersecurity Research and Applied Software Engineering. I made sure to follow the rules set by the University of the West of Scotland so my study of OWASP Sentinel was based on evidence and research that had been carefully reviewed by other experts.

What I learned from this project is that security is really important the idea of "Shift-Left"

security. I found out that if you use automated diagnostics in a way they can be just as good, as the tools that big companies use, even when you do not have a lot of resources.

8.5 Full Technical Roadmap for Future Iterations

The current version of OWASP Sentinel does what it is supposed to do it meets all the requirements that were listed when the project started. But the internet is always changing so OWASP Sentinel needs a plan, for what to do.

8.5.1 Headless Browser and DOM Logic Integration

Modern Single Page Applications need more than HTTP parsing. They need a DOM execution environment.

The tool will work with Playwright or Puppeteer libraries in the future. This will help the scanner do things like click buttons fill out forms and wait for events on web pages before checking the state.

This improvement will help the tool find security issues in modern React and Vue.js applications.

The scanner will be able to Act like an User and that will make it better, at finding problems.

It will cover areas where attacks can happen.

Modern web applications use React and Vue.js a lot. This is important.

8.5.2 Advanced Heuristic AI/ML Analysis Layer

As we talked about in Chapter 2 the next thing that makes sense to do is add an AI-driven layer. If we use a Transformer-based model the Differential Analyzer can learn to tell the difference, between Benign Dynamic Content and Malicious Reflection almost as well as a human can. To do this we need to teach the system using thousands of labeled results so the Confidence Metric becomes a really smart predictive score. The Differential Analyzer will get better at figuring things out. We will use the

Confidence Metric to make the Differential Analyzer even smarter.

8.5.3 API-Centric Scanning and Protocol Fuzzing

With the rise of apps and serverless architectures the focus of web security has moved to APIs. Future work will include a module for GraphQL and RESTful Schema Analysis. This module will automatically check for "Object Level Authorization" (BOLA) and other API risks in the OWASP API Security Top 10. This way OWASP Sentinel stays a security tool for modern web with many protocols.

In short OWASP Sentinel is not a product but a strong and flexible starting point for new context-aware security checks that are easy to use.

OWASP Sentinel is not a product but a strong and flexible starting point, for new context-aware security checks that are easy to use.

Appendices

A.1 System Directory Structure

OWASP-Sentinel/

```
|
|├─ core/
| |├─ context.py
| |├─ crawler.py
| |├─ cvss.py
| |├─ database.py
| |├─ diffing.py
| |├─ logger.py
| |├─ parser.py
| |├─ session.py
| |└─ utils.py
```

```
|
|├─ modules/
| |├─ sqli.py
| |└─ xss.py
```

- | └─ ssti.py
- | └─ lfi.py
- | └─ host_header.py
- | └─ misconfig.py
- | └─ open_redirect.py
- | └─ techdetect.py

- |
- | └─ reporting/
- | | └─ html_report.py
- | | └─ json_report.py
- |

- └─ └─ engine.py
- └─ └─ sentinel.py

Appendix B: References

- OWASP Foundation. (2025). *OWASP Top 10 – Web Application Security Risks*. Available at: <https://owasp.org/>
- Sommerville, I. (2016). *Software Engineering* (10th ed.). Pearson.
- Kizza, J. (2017). *Guide to Computer Network Security*. Springer
- Bau, J., et al. (2010). *State of the Art: Automated Black-Box Web Application Vulnerability Testing*. IEEE Symposium.
- Antunes, N., & Vieira, M. (2015). *Comparing the Effectiveness of Vulnerability Scanners*. IEEE Transactions.
- Vigna, G. (2013). *The Economics of Vulnerability Disclosure*.
- NIST. (2023). *Vulnerability Management Lifecycle Framework*.
- OWASP Juice Shop Project. Available at: <https://owasp.org/www-project-juice-shop/>
- Damn Vulnerable Web Application (DVWA). Available at: <http://www.dvwa.co.uk/>

B.1 Tools & Technologies Referenced

- Python 3.x Documentation

- Asyncio Library (Python Docs)
- SQLite Documentation
- BeautifulSoup4 Documentation
- Requests / urllib3 Documentation

Appendix C: Project Meetings & Progress Log

C.1 Meeting Log Table (Oct 2025 – Apr 2026)

Meeting No.	Date	Participants	Discussion Summary	Outcome
1	Oct 2025	Student, Supervisor	Initial project proposal (OWASP Sentinel idea)	Topic approved
2	Nov 2025	Student, Supervisor	Literature review direction & research scope	Scope refined
3	Dec 2025	Student	Requirements gathering (interviews)	Requirements finalized

			& analysis)	
4	Jan 2026	Student, Supervisor	System architecture & design review	Design approved
5	Feb 2026	Student	Core implementation (engine + modules development)	Core system built
6	Mar 2026	Student, Supervisor	Testing strategy & evaluation planning	Testing framework finalized
7	Apr 2026	Student	Final implementation + report writing	Draft completed
8	Apr 2026	Student, Supervisor	Final review, feedback & corrections	Ready for submission

C.2 Monthly Progress Breakdown

Month	Work Completed
-------	----------------

Oct 2025	Topic selection & proposal submission
Nov 2025	Literature review & research gap identification
Dec 2025	Interim Report Submission
Jan 2026	System design & architecture
Feb 2026	Core implementation (modules + engine) & Presentation
Mar 2026	Dessertation Progress
Apr 2026	Report writing, review & final submission

C.3 Key Challenges & Solutions

Challenge	Solution
Handling HTTP 429 / 503 errors	Implemented exponential backoff retry mechanism
High false positives	Introduced differential response analysis
Performance issues during scanning	Integrated asyncio-based concurrency
Difficulty with SPA (React/Vue apps)	Focused on API-level and backend testing